

Experiment No.04

Name of Experiment: Write a program to implement Round Robin Algorithm.

Problem Statement: Write a program in C to implement Round Robin Scheduling algorithms

Software /Hardware Requirements: Turbo C++ Compiler, Personal Computer

Theory:

Round Robin (RR) is a preemptive CPU scheduling algorithm where each process is assigned a fixed time slot (Time Quantum) in a cyclic manner.

It is one of the simplest and most widely used scheduling techniques, especially in time-sharing systems.

Working Principle:

1. All processes are kept in a **ready queue**.
2. CPU picks the first process and gives it the **CPU for a fixed time quantum**.
3. If the process **finishes within the time**, it is removed from the queue.
4. If it **does not finish**, it is preempted and placed at the **back of the queue** with its **remaining**
5. **burst time**.
6. This cycle repeats until all processes are completed.

Key Terms:

- **Time Quantum (TQ):**
The fixed time period a process can execute before being preempted.
- **Context Switching:**
Saving and loading the state of a process when switching between processes.

Formulas:

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time

Program Code:

/* Write a program in C to implement Round Robin Scheduling Algorithm*/

```
#include <stdio.h>
struct Process {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int waiting;
    int turnaround;
};
```

```
int main() {
    int n, time = 0, quantum, completed = 0;
    struct Process p[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input arrival and burst times
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for Process %d: ",
p[i].pid);
        scanf("%d%d", &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
        p[i].waiting = 0;
        p[i].turnaround = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);

    int done;
    do {
        done = 1;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0 && p[i].arrival <= time) {
                done = 0;
                if (p[i].remaining > quantum) {
                    time += quantum;
                    p[i].remaining -= quantum;
                } else {
                    time += p[i].remaining;
                    p[i].waiting = time - p[i].arrival - p[i].burst;
                    p[i].turnaround = time - p[i].arrival;
                    p[i].remaining = 0;
                    completed++;
                }
            } else if (p[i].arrival > time && p[i].remaining > 0) {
                // If no process has arrived, skip time forward
                time++;
                break;
            }
        }
    } while (completed < n);

    // Output results
    float total_wait = 0, total_turn = 0;
    printf("\nPID\tArrival\tBurst\tWaiting\tTurnaround\n");
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].arrival, p[i].burst, p[i].waiting,
p[i].turnaround);
        total_wait += p[i].waiting;
        total_turn += p[i].turnaround;
    }

    printf("Average Waiting Time: %.2f\n", total_wait / n);
    printf("Average Turnaround Time: %.2f\n", total_turn / n);

    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0 5
Enter Arrival Time and Burst Time for Process 2: 1 3
Enter Arrival Time and Burst Time for Process 3: 2 1
Enter Time Quantum: 2
```

PID	Arrival	Burst	Waiting	Turnaround
1	0	5	4	9
2	1	3	4	7
3	2	1	2	3

Average Waiting Time: 3.33

Average Turnaround Time: 6.33

Conclusion: In this way we implement a program in C for Round Robin Scheduling algorithms.

Experiment No.05

Name of Experiment: Write a program to implement Producer Consumer Problem

Problem Statement: Write a program in C to implement producer Consumer Problem

Software /Hardware Requirements: GCC Compiler, Personal Computer

Theory:

The Producer-Consumer problem is a classic synchronization issue in operating systems. It involves two types of processes: producers, which generate data, and consumers, which process that data. Both share a common buffer. The challenge is to ensure that the producer doesn't add data to a full buffer and the consumer doesn't remove data from an empty buffer while avoiding conflicts when accessing the buffer. In this article, we are going to solve this problem by using semaphores.

What is a Semaphore?

A semaphore is a synchronization tool used in computing to manage access to shared resources. It works like a signal that allows multiple processes or threads to coordinate their actions. Semaphores use counters to keep track of how many resources are available, ensuring that no two processes can use the same resource at the same time, thus preventing conflicts and ensuring orderly execution.

A **semaphore** *S* is an integer variable that can be accessed only through two standard operations: `wait()` and `signal()`. The `wait()` operation reduces the value of the semaphore by 1 and the `signal()` operation increases its value by 1.

```
wait(S){
while(S<=0);    // busy waiting
S--;
}

signal(S){
S++;
}
```

Types of Semaphore

- **Binary Semaphore** - This is similar to mutex lock but not the same thing. It can have only two values - 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problems with multiple processes.
- **Counting Semaphore** - Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Producer Consumer Problem Statement

We have a buffer of fixed size. A producer can produce an item and can place it in the buffer. A consumer can pick items and consume them. We need to ensure that when a producer is placing an item in the buffer, then at the same time consumer should not consume any item. In this problem, the buffer is the critical section.

To solve this problem, we need two counting semaphores - Full and Empty. "Full" keeps track of some items in the buffer at any given time and "Empty" keeps track of many unoccupied slots.

Initialization of semaphores

`mutex = 1`

`Full = 0` // Initially, all slots are empty. Thus full slots are 0

`Empty = n` // All slots are empty initially